

Synthetic Dataset Construction Pipeline

Contents

1	Pipeline Core Design	1
1.1	Current MVP	2
1.2	Literature overview	3
1.2.1	Generic papers	3
1.2.2	Clinical papers and MIMIC-IV role	4
2	Pipeline	4
2.1	Stage 1: Ontology and Configuration	4
2.2	Stage 2: Hidden Query Plans	5
2.3	Stage 3: Dominance Calibration	6
2.4	Stage 4: Structured Clinical Facts	6
2.5	Stage 5: Chunk Rendering and Normalization	7
2.6	Stage 6: Queries and Canonical Answers	8
2.7	Stage 7: Qrels from Hidden Memberships	9
2.8	Stage 8: Embeddings	9
2.9	Stage 9: Geometry Filtering	9
2.10	Stage 10: Retrieval Evaluation	10
2.11	Stage 11: Diagnostics and Plots	10
3	Data splits	11
4	Prompt and Template Families	11
5	Metrics	12
6	Benchmark Objects	13

1 Pipeline Core Design

The current implementation builds the benchmark from hidden structure before any retrieval text exists. This is the important reversal from the old MIMIC-IV-only workflow: the code first creates a controlled multi-facet retrieval problem, then renders it into clinical prose, embeds it, filters by geometry, and finally evaluates retrieval.

1. load a controlled ontology of conditions, subgroup labels, and two clinical axes;
2. enumerate hidden query plans with four required answer facets;
3. optionally calibrate which facet should be overrepresented using probe embeddings;
4. generate structured clinical fact rows for gold facets, near-miss distractors, and background outlier clusters;
5. render fact rows into validated clinical-style chunk documents;
6. render deterministic natural-language queries and canonical gold answers;
7. derive qrels directly from the hidden chunk-facet membership table;

8. embed queries and chunks, filter queries by coverage-sensitive embedding geometry, and evaluate top- k , MMR, and facility-location.

The implementation lives under:

```
src/experiments/medical_dataset_gen
```

The pipeline entry point is:

```
uv run python -m experiments.medical_dataset_gen.run_pipeline --exp <experiment>
```

The experiment configuration must already exist at:

```
src/experiments/medical_dataset_gen/_results/<experiment>/_config.yaml
```

The code intentionally no longer reads a source-level `_config.yaml` or copies a default config. CLI flags only override selected fields of the per-experiment config. The default ontology is `ontology.yaml`, unless `generation.ontology_path` points to a different YAML file.

1.1 Current MVP

The current MVP is small but no longer just a sketch. The executable pipeline is defined by `run_pipeline.py`:

Stage name	Function	Main output
plans	<code>run_make_query_plans()</code>	<code>query_plans.parquet</code>
calibrate_plans	<code>run_calibrate_query_plans()</code>	<code>query_plans.parquet</code> , <code>query_plan_calibration.parquet</code>
facts	<code>run_make_facts()</code>	<code>clinical_facts.parquet</code>
chunks	<code>run_make_chunks()</code>	<code>chunk_documents.parquet</code> , <code>chunk_memberships.parquet</code> , <code>generation_rejects.parquet</code>
queries_answers	<code>run_make_queries_answers()</code>	<code>queries.parquet</code> , <code>gold_answers.parquet</code>
qrels	<code>run_make_qrels()</code>	<code>qrels.parquet</code>
embed	<code>run_embed()</code>	embedding memmaps and <code>embeddings_metadata.json</code>
geom_filter	<code>run_filter_geometry()</code>	<code>geometry_stats.parquet</code>
eval	<code>run_evaluate()</code>	<code>evaluation_results.parquet</code> , <code>evaluation_stats.parquet</code>
geom_plots	<code>run_embedding_geometry()</code>	embedding-geometry parquet diagnostics and figures
eval_plots	<code>store_eval_figures()</code>	evaluation figures

Stages can be sliced with the implemented CLI flags:

```
uv run python -m experiments.medical_dataset_gen.run_pipeline --exp mvp --only chunks
uv run python -m experiments.medical_dataset_gen.run_pipeline --exp mvp --from embed
uv run python -m experiments.medical_dataset_gen.run_pipeline --exp mvp --from
geom_filter --to eval_plots
```

The default benchmark focus is `retrieval.pool_scope=query_local`. In this setting the candidate pool for a query is the controlled local universe generated for that query: gold chunks, hard near-miss

distractors, and any configured background outlier clusters. The code also supports `same_condition` and `full_corpus`, but the current thesis mechanism is easiest to diagnose in the query-local pool.

The implemented MVP has these constraints:

- each query has exactly four answer facets: two compared subgroups crossed with `treatment_duration` and `rehab_outcome`;
- supported query types are `subgroup_comparison` and `outcome_synthesis`;
- the ontology may contain many conditions and subgroups, but each run selects the first `global.conditions` conditions in ontology order;
- one facet is intentionally dominant, receiving `generation.gold_chunks_dominant` fact rows, while the other facets receive `generation.gold_chunks_complementary`;
- hard distractors cycle through same-condition wrong-subgroup, same-subgroup wrong-condition, and same-axis wrong-condition patterns;
- optional background outlier clusters create compact but irrelevant clinical islands inside the query-local pool;
- chunk text can be LLM-generated, deterministic template text, or deterministic text rewritten by an LLM;
- queries are deterministic template renderings, with optional LLM paraphrasing guarded by entity and axis checks;
- gold answers are deterministic summaries of the hidden fact rows, not LLM-authored answers.

No Prolog solver, agentic retrieval, information-not-found cases, conflicting-note resolution, or LLM-as-judge qrel creation is part of the current reduced implementation. Those remain possible extensions, but they should not be described as current pipeline stages.

1.2 Literature overview

1.2.1 Generic papers

The papers start from an already existing data source and build a QA benchmark on top of it. The main ideas in the papers are: breaking down the data source into structured data supporting knowledge discovery and query generation, allowing control of query difficulty, explicit validation of synthetic data, explicit evidence provenance, and diversity checks.

This pipeline borrows their construction methods, but changes the starting point: MIMIC-IV is only historical context and a possible reference for terminology, condition families, comorbidity co-occurrences, demographic bins, and note style. The actual benchmark starts from a hidden schema, so that the retrieval problem has the multi-cluster structure that the raw discharge-note corpus did not reliably expose.

Useful ideas from the literature:

- **Evidence-first construction.** MHTS builds questions from claims, clusters, evidence mappings, and difficulty signals [9]. DRAGON stores explicit links among documents, queries, clues, answers, and citations [12]. In this pipeline, those ideas become `query_plans.parquet`, `clinical_facts.parquet`, `chunk_documents.parquet`, `chunk_memberships.parquet`, and `qrels.parquet`. Text generation can use an LLM, but gold labels come from the hidden mapping.
- **Multi-aspect query structure.** ClaimSpect and MQ4CS are useful because they treat a broad user query as a set of separable aspects [6, 1]. Here that decomposition is used before generation: a query plan explicitly chooses facets such as subgroup, treatment duration, and rehabilitation outcome.
- **Question-aware corpus construction.** EnterpriseRAG-Bench generates documents around planned question structures, cross-document coherence, realistic noise, completeness clusters, conflicting near-duplicates, answer facts, and conformance checks [13]. The current reduced

implementation uses the relevant core: planned query-local clusters, intentionally redundant gold evidence, hard distractors, and embedding-geometry filtering.

- **On-demand hidden worlds.** PhantomWiki generates a fresh fictional universe from hidden symbolic facts, then derives templated questions and exhaustive answers with a solver [4]. Here the equivalent is a deterministic clinical world with fixed seeds, hidden fact tables, JSON logical forms, template-rendered queries, canonical answers, and qrels derived before retrieval.
- **Difficulty control.** MHTS motivates storing difficulty as number of facets, number of evidence pieces, and query–evidence similarity [9]. In this benchmark, difficulty is controlled by `n_facets`, dominant and complementary gold counts, distractor counts, background outlier clusters, and whether *top-k* collapses onto the planned dominant facet.
- **Validation and provenance.** Rahmani et al. and RAGVAL support automatic test-collection construction but also show why validation matters [11, 7]. This pipeline therefore stores fact IDs, facet IDs, cluster IDs, membership rows, reject rows, and geometry-filter diagnostics.
- **Diversity.** DATAGEN uses attribute-guided generation, diversity-oriented example selection, and group checking to avoid duplicate synthetic examples [5]. Driouich et al. use a diversity agent that clusters and samples source documents for broader RAG coverage [3]. Those methods cannot be copied literally, because this benchmark needs both diversity and planned redundancy. The goal here is to preserve redundancy inside a dominant gold facet, enforce separation among gold facets, and create distractors that are close enough to challenge MMR.
- **Downstream filtering.** Kim et al. argue that synthetic generators should be judged by the downstream behavior their data induces [8]. For the thesis, a generated item is useful only if the candidate pool actually exposes a coverage-sensitive retrieval problem.
- **Optional prompt optimization.** If prompts are unstable, prompt selection can be treated as a multi-objective optimization problem. Pastroián et al. provide one example of automatic prompt evolution through evolutionary multi-objective prompt learning [10]. This is optional and not part of the current executable stage graph.

1.2.2 Clinical papers and MIMIC-IV role

MIMIC-IV can still be used as a reference for terminology, frequent conditions, comorbidity distributions, age groups, and discharge-note style, but it should not drive the current qrels. The qrels are generated from the hidden structured construction.

This can be strengthened with ideas from the medical papers for synthetic data construction:

- simple care-path templates that say, for a given condition and subgroup, which treatments and outcomes are plausible. This follows the same general idea as HealthBranches [2];
- a small controlled vocabulary of medical terms, so the generator does not randomly alternate between inconsistent names for the same condition, treatment, or outcome. This is the practical part of knowledge-infused medical QA systems such as KI-MAG [14];
- explicit patient-group rules, such as “encephalitis + age over 75 + rehabilitation outcome” or “pneumonia + diabetes + treatment duration”. This borrows the idea of decomposing clinical questions into condition, subgroup, treatment, and outcome constraints [15].

2 Pipeline

2.1 Stage 1: Ontology and Configuration

The pipeline starts from `ontology.yaml` and the experiment config stored in `_results/<experiment>/_config.yaml`. The ontology has three top-level blocks:

- **conditions:** display names, search terms, treatments, treatment-duration bins, and rehabilitation-outcome bins;
- **subgroups:** demographic and comorbidity labels, fields, values, and aliases;

- `clinical_axes`: currently exactly `treatment_duration` and `rehab_outcome`.

The code validates the ontology with Pydantic models in `generation/schemas.py`. The MVP explicitly requires the two clinical axes to be exactly `treatment_duration` and `rehab_outcome`. A representative condition entry is:

```
encephalitis_myelitis:
  display: encephalitis or myelitis
  terms: [encephalitis, myelitis, neurologic deficit, acyclovir]
  treatments: [acyclovir, corticosteroids, supportive care]
  duration_treatments: [acyclovir, corticosteroids]
  duration_days:
    short: [5, 7]
    standard: [10, 14]
    prolonged: [21, 28]
  rehab_outcomes:
    home_rehab:
      - discharged home with outpatient neurorehabilitation
    inpatient_rehab:
      - transferred to inpatient rehabilitation for persistent weakness
  persistent_deficit:
    - persistent focal deficits requiring close neurologic follow-up
```

Important config fields include:

- `global.n_queries`, `global.conditions`, and `global.seed`;
- `generation.query_types`, `generation.dominance_mode`, gold chunk counts, distractor counts, and background outlier settings;
- `generation.use_llm_chunk_generation`, `generation.use_llm_chunk_rewriting`, and `generation.use_llm_query_paraphrase`;
- embedding model and device settings under `embeddings`;
- candidate-pool scope, k values, λ values, and MMR window under `retrieval`;
- pass/fail thresholds under `geometry`.

2.2 Stage 2: Hidden Query Plans

`generation/query_plans.py` creates `query_plans.parquet`. It loads the ontology, selects the first `global.conditions` condition entries, enumerates every unordered subgroup pair, and emits plans round-robin over conditions so that the dataset is not filled by one condition prefix.

Each plan has two subgroups and two axes. This produces four required facets:

1. subgroup A + `treatment_duration`;
2. subgroup A + `rehab_outcome`;
3. subgroup B + `treatment_duration`;
4. subgroup B + `rehab_outcome`.

The plan stores both scalar columns and JSON columns. `facets_json` contains serialized `QueryPlanFacet` objects. `logical_form_json` contains condition, subgroup IDs, axis IDs, facet IDs, and the dominant facet ID. This is ordinary JSON, not a Prolog layer.

Example plan shape:

```
query_id: q00001
query_type: subgroup_comparison
template_id: subgroup_comparison
condition_id: encephalitis_myelitis
subgroup_a_id: age_over_75
subgroup_b_id: uncomplicated_diabetes
```

```

dominant_facet_id: q00001_f1
n_facets: 4
facets:
  - facet_id: q00001_f1
    subgroup_id: age_over_75
    axis: treatment_duration
    value_bin: short
    cluster_id: q00001_c1
    cluster_role: dominant_gold
    target_gold_chunks: 18
  - facet_id: q00001_f2
    subgroup_id: age_over_75
    axis: rehab_outcome
    cluster_role: complementary_gold
    target_gold_chunks: 8

```

The split assignment is deterministic from the plan index: three buckets out of twenty go to test, three to validation, and the rest to train. Query-template variants are also assigned deterministically in round-robin order within each query type.

2.3 Stage 3: Dominance Calibration

`generation/calibrate_plans.py` creates `query_plan_calibration.parquet`. It may also rewrite `query_plans.parquet`.

There are two modes:

- **dominance_mode=rotating**: keep the dominant facet assigned during plan generation and write a calibration audit table;
- **dominance_mode=embedding_calibrated**: render neutral query text and neutral deterministic probe chunks for each facet, embed them, and choose the facet that is naturally closest to the query.

The calibrated mode uses `generation.dominance_probe_chunks_per_facet` deterministic probe chunks per facet. For each facet it computes query-similarity statistics such as mean, p25, p50, and p75. The selected facet maximizes the p25-over-best-complement-p75 margin, with mean query similarity as a tie-breaker. The stage then updates:

- `dominant_facet_id`;
- each facet's `cluster_role`;
- each facet's `target_gold_chunks`;
- the logical form's dominant facet field.

This stage is important because it keeps the natural query wording neutral. The dominant facet is not exposed through an artificial anchor phrase; it is chosen from the embedding geometry that the normal query already induces.

2.4 Stage 4: Structured Clinical Facts

`generation/facts.py` creates `clinical_facts.parquet`. For every query plan, it expands each facet into hidden fact rows. These rows are the source of truth for later chunks, qrels, and answers.

Gold fact rows have:

- `is_gold=true`;
- `facet_id` equal to the facet they support;
- `target_facet_id` equal to the same facet;
- `cluster_role` equal to `dominant_gold` or `complementary_gold`.

Hard distractor rows have:

- `is_gold=false`;
- `facet_id=null`;
- `target_facet_id` set to the facet they are meant to confuse;
- `cluster_role=hard_distractor`;
- `distractor_type` in `same_condition_wrong_subgroup`, `same_subgroup_wrong_condition`, or `same_axis_wrong_condition`.

Background outlier rows have:

- `is_gold=false`;
- no facet or target facet;
- `cluster_role=background_outlier`;
- `distractor_type=background_clinical_cluster`.

Example fact:

```
fact_id: q00001_g_9_000_1234
query_id: q00001
chunk_reuse_key: <sha256>
facet_id: q00001_f3
target_facet_id: q00001_f3
cluster_id: q00001_c3
cluster_role: complementary_gold
condition_id: encephalitis_myelitis
subgroup_id: uncomplicated_diabetes
axis: treatment_duration
value_bin: prolonged
duration_days: 24
treatment: acyclovir
is_gold: true
must_mention:
  - encephalitis or myelitis
  - patients with uncomplicated diabetes
  - "24"
  - treatment duration
```

The code validates axis-specific fields. Duration facts must have `duration_days` and `treatment`, and must not have `rehab_outcome`. Rehabilitation facts must have `rehab_outcome`, and must not have `duration_days` or `treatment`.

A key implementation detail is `chunk_reuse_key`. It is derived from condition, subgroup, axis, value bin, local index, and reuse scope, but not from `query_id`. Equivalent structural facts can therefore reuse the same surface chunk across queries. This is the current mechanism for controlled redundancy and generation-cache reuse.

2.5 Stage 5: Chunk Rendering and Normalization

`generation/chunks.py` creates `chunk_documents.parquet`, `chunk_memberships.parquet`, and `generation_rejects.parquet`. The old denormalized `chunks.parquet` is no longer the current artifact; if it exists, the chunk stage removes it.

There are three rendering modes:

1. **Direct LLM generation:** `use_llm_chunk_generation=true`. The LLM writes each chunk under strict constraints, with validation and retry feedback.
2. **Deterministic templates:** both LLM chunk flags are false. The code renders chunk text from YAML template inventories under `generation/templates_data`.

3. **Template** `rewrite:` `use_llm_chunk_generation=false` and `use_llm_chunk_rewriting=true`. The deterministic draft is rewritten by the LLM; if rewriting fails validation, the deterministic draft is kept.

Validation is implemented in `generation/chunk_templates.py`. Hard errors reject the chunk and drop the whole query, because a kept query must have complete local evidence. Hard checks include hidden benchmark terms, leading note-section headers, missing condition evidence, missing subgroup evidence, missing treatment duration days, missing treatment name, and missing rehabilitation-outcome evidence. Soft warnings, such as a duration chunk containing rehabilitation language, are stored but do not necessarily reject the chunk.

Example rendered chunk:

```
The 82-year-old woman older than 75 was admitted with encephalitis or myelitis and new gait instability. For total treatment duration, acyclovir was continued for 21 days. Neurologic symptoms improved slowly, and the longer course was documented because deficits persisted near discharge.
```

The stage normalizes rows into two tables:

- `chunk_documents.parquet`: one retrievable document per `chunk_reuse_key`, with `chunk_id`, `text`, `generation` metadata, and clinical surface fields;
- `chunk_memberships.parquet`: query-specific links from `query_id`, `fact_id`, `facet_id`, `target_facet_id`, `cluster_id`, `cluster_role`, and `is_gold` to a document `chunk_id`.

This split matters. Retrieval embeds unique chunk documents, while qrels and facet coverage are query-local through the membership table.

2.6 Stage 6: Queries and Canonical Answers

`generation/queries_answers.py` creates `queries.parquet` and `gold_answers.parquet`. It first reads `generation_rejects.parquet`; any query dropped during chunk rendering is skipped.

Query `text` is rendered by `generation/query_templates.py` from `generation/templates_data/query_templates.yaml`. The currently implemented query types are:

- `subgroup_comparison`: asks how treatment duration and rehabilitation outcome differ between two subgroups;
- `outcome_synthesis`: asks for therapy-course length and discharge rehabilitation status for subgroup A versus subgroup B.

Example query:

```
For patients diagnosed with encephalitis or myelitis, how do treatment duration and rehabilitation outcome differ between patients older than 75 and patients with uncomplicated diabetes?
```

If `generation.use_llm_query_paraphrase=true`, an LLM may paraphrase the deterministic query. The paraphrase is accepted only if it still contains the condition display, both subgroup labels, and language for both axes. Otherwise the deterministic query is kept.

The canonical answer is deterministic. The code groups gold fact rows by facet, computes facet summaries, and writes a compact four-part answer. Duration summaries include modal value bin, average duration in days, and common treatment. Rehabilitation summaries include modal value bin and common outcome phrase.

Example answer object:


```

query_id: q00001
answer_text: >
  For patients older than 75, the synthetic corpus shows a short course,
  averaging 6.1 days, most often with acyclovir for treatment duration and
  an inpatient rehab pattern, commonly described as transferred to inpatient
  rehabilitation for persistent weakness for rehabilitation outcome. For
  patients with uncomplicated diabetes, it shows a prolonged course,
  averaging 24.0 days, most often with acyclovir for treatment duration and
  a home rehab pattern, commonly described as discharged home with outpatient
  neurorehabilitation for rehabilitation outcome.
facet_summaries_json: {"q00001_f1": "...", "q00001_f2": "..."}
answer_facts_json: [{"facet_id": "q00001_f1", "supporting_fact_ids": [...]}]

```

2.7 Stage 7: Qrels from Hidden Memberships

`generation/qrels.py` creates `qrels.parquet` from `chunk_memberships.parquet`. The rule is intentionally simple:

```

is_gold = true  -> relevance_grade = 1, support_type = positive
is_gold = false -> relevance_grade = 0

```

Non-gold chunks are further labeled as `hard_negative` or `background_outlier`. The qrels preserve:

```

query_id, chunk_id, fact_id, facet_id, target_facet_id, cluster_id,
cluster_role, is_gold, distractor_type, relevance_grade, support_type

```

This stage is fundamental to the benchmark design. Relevance is not inferred from generated prose, lexical overlap, embedding similarity, LLM judgement, or human annotation. It is projected from the hidden construction.

2.8 Stage 8: Embeddings

`retrieval/embed.py` creates embedding memmaps for unique chunk documents and queries. Chunk text is read from `chunk_documents.parquet`; query text is read from `queries.parquet`. The implementation uses `helpers.embedder.Embedder`, which wraps SentenceTransformers-style document and query embedding calls.

The output files are:

- `embeddings_chunk_vectors.npy`;
- `embeddings_query_vectors.npy`;
- `embeddings_chunk_ids.npy`;
- `embeddings_query_ids.npy`;
- `embeddings_metadata.json`.

With `embeddings.normalize=true`, the dot product used by later stages is cosine similarity:

```

sim(query, chunk) = e_query dot e_chunk

```

If multiple devices are provided under `embeddings.devices`, the embedder uses SentenceTransformers multi-process encoding. The same embedding configuration is also used by dominance calibration.

2.9 Stage 9: Geometry Filtering

`retrieval/filter_geometry.py` creates `geometry_stats.parquet`. This stage asks whether a generated query actually exposes the intended retrieval geometry. It uses hidden labels for dataset validation, not as features exposed to retrieval methods.

For each query, the stage:

1. builds the configured candidate pool: `query_local`, `same_condition`, or `full_corpus`;
2. keeps the top `retrieval.candidate_pool_n` candidates by query similarity;
3. checks whether every hidden gold facet appears in that top- N pool;
4. checks whether nearest-neighbor top- k over-selects the planned dominant facet;
5. checks whether top- k already retrieves too many distinct facets;
6. checks whether in-facet similarity exceeds cross-facet similarity by the configured margin;
7. checks whether enough near-miss distractors appear in the candidate pool;
8. checks whether configured background outlier clusters are present and complete.

The current pass condition is:

```
all facets present in top-N
planned_topk_dominant_count >= geometry.min_topk_dominant_count
n_topk_retrieved_facets <= geometry.max_topk_retrieved_facets
mean_in_facet_similarity - mean_cross_facet_similarity
    >= geometry.min_in_minus_cross_similarity
n_near_miss_distractors_in_pool >= geometry.min_distractors_in_pool
background outliers complete when enabled
```

The field `retrieval.only_pass_geometry` controls whether evaluation skips failed queries. The evaluation stage also checks that `geometry_stats.pool_scope` matches the current retrieval config, because changing pool scope changes the meaning of the geometry filter.

2.10 Stage 10: Retrieval Evaluation

`evaluation/evaluate.py` creates `evaluation_results.parquet` and `evaluation_stats.parquet`. It compares:

- `top_k`: nearest-neighbor baseline;
- `mmr`: maximal marginal relevance, a dispersion method;
- `fac_loc`: greedy facility-location selection, the coverage method of interest.

All methods operate inside the same semantic top- N candidate pool. Top- k is evaluated once per configured k . MMR and facility-location are evaluated for each configured λ value.

The facility-location selector uses:

```
score(j) = lambda * sim(q, j)
          + (1 - lambda) * marginal_coverage_gain(j)
```

The expected thesis behavior is:

- top- k may retrieve many relevant chunks but repeatedly from the dominant facet;
- MMR may reduce redundancy but sometimes spends budget on semantically distant distractors;
- facility-location should prefer representatives of uncovered local clusters when the generated geometry is successful.

2.11 Stage 11: Diagnostics and Plots

The final two executable stages are diagnostic:

- `embedding_geometry/run.py` creates `embedding_geometry_points.parquet`, `embedding_geometry_query_stats.parquet`, and per-query UMAP/PCA/HDBSCAN figures under `_figures/embedding_geometry`;
- `evaluation/plots.py` creates evaluation figures under `_figures/evaluation`.

Embedding-geometry diagnostics rebuild the candidate pool for representative queries, label candidates by hidden facet or distractor type, overlay top- k /MMR/facility-location selections, and compute cluster diagnostics. The automatic query selection can use a mixed set of good, mid, and bad cases, or explicit query IDs from the config.

Evaluation plots treat top- k as the baseline. For MMR and facility-location, many comparison plots choose a coverage-first λ^* path: maximize mean `FacetCoverage@k`; break ties by higher precision, lower distractor rate, and then higher `alpha-nDCG@k`.

3 Data splits

Splits are assigned deterministically from the plan index:

- **train**: default split for most generated plans;
- **validation**: used to inspect generation quality and choose retrieval hyperparameters such as MMR/facility-location λ values;
- **test**: final held-out evaluation.

The current evaluation code reports over whatever queries are present after generation rejects and geometry filtering. Split-specific reporting can be added downstream if needed, because each query row keeps its `split`.

4 Prompt and Template Families

Stage	Prompt/template family	Current role
Query plans	Query template IDs	No LLM. <code>query_plans.py</code> assigns stable template IDs in round-robin order.
Clinical facts	Structured ontology sampling	No LLM. Facts are generated by seeded sampling over condition bins, subgroup labels, and axis-specific values.
Chunks	Chunk templates or LLM chunk prompt	Direct LLM generation is the default in example configs; deterministic templates are used for fast smoke tests; rewrite mode keeps deterministic fallback on failure.
Queries	Query templates with optional paraphrase prompt	Deterministic YAML templates are primary. LLM paraphrases are accepted only if required entities and axes remain present.
Gold answers	Deterministic aggregation	No LLM. Answers are synthesized from grouped gold facts and stored with facet summaries and supporting fact IDs.
Qrels	Hidden membership projection	No LLM. Relevance labels are derived from <code>is_gold</code> in <code>chunk_memberships.parquet</code> .

5 Metrics

Metric	Purpose
FacetCoverage@k	Fraction of hidden answer facets represented by at least one selected gold chunk. This is the main thesis metric.
MeanFacetRecall@k	Mean per-facet recall. This is the current summary label for the raw <code>weighted_facet_coverage</code> field.
Precision@k	Fraction of selected chunks that are gold for any answer facet.
Recall@k	Fraction of all gold chunks retrieved, regardless of facet. Useful but secondary because the dataset intentionally contains many redundant gold chunks.
MAP@k	Conventional average precision over binary gold labels. Secondary for the same reason as recall.
FacetMRR@k	Mean reciprocal rank of the first selected evidence chunk for each facet.
alpha-nDCG@k	Diversified ranking score using <code>facet_id</code> as the subtopic label and diminishing gain for repeated facets.
AnswerROUGE1Recall@k	Lexical recall between selected chunk text and the canonical answer after query-term filtering.
MacroFacetAnswerROUGE1Recall@k	Average answer-overlap recall over facet-level reference summaries.
DistractorRate	Fraction of selected chunks that are non-gold. Lower is better.
NearMissDistractorRate	Fraction of selected chunks from facet-like hard negatives.
BackgroundOutlierRate	Fraction of selected chunks from coherent non-gold background islands.
DominantFacetRate	Fraction of selected chunks whose facet equals the planned dominant facet. High values indicate collapse onto the overrepresented facet.
RedundantGoldRate	Fraction of selected chunks that are gold but do not add a new facet.
fac	Facility-location objective score of the selected set in embedding space. Diagnostic, not a qrel metric.
avg_cos	Average query cosine similarity of selected chunks.
jac	Jaccard overlap between a method’s selected set and the top- <i>k</i> selected set.

6 Benchmark Objects

Artifact	Purpose
<code>query_plans.parquet</code>	Hidden plan for each query: condition, compared subgroups, query type, template ID, facets, dominant facet, logical form, split, and target gold counts.
<code>query_plan_calibration.parquet</code>	Facet table for rotating or embedding-calibrated dominant-facet selection.
<code>clinical_facts.parquet</code>	Hidden structured source of truth: fact IDs, chunk reuse keys, condition/subgroup/axis fields, concrete duration or rehab values, gold/distractor labels, and validation constraints.
<code>chunk_documents.parquet</code>	Retrieval corpus of unique chunk documents: <code>chunk_id</code> , text, clinical surface fields, generation source, and validation-warning metadata.
<code>chunk_memberships.parquet</code>	Query-specific mapping from facts and facets to chunk documents: <code>query_id</code> , <code>chunk_id</code> , <code>fact_id</code> , <code>facet_id</code> , <code>cluster_role</code> , <code>is_gold</code> , and <code>distractor_type</code> .
<code>generation_rejects.parquet</code>	Chunk-generation failures; any query with a reject is skipped by the query/answer stage.
<code>queries.parquet</code>	User-facing query text plus condition, query type, template ID, split, facets JSON, and logical form JSON.
<code>gold_answers.parquet</code>	Canonical deterministic answer text, facet summaries, answer fact objects, supporting fact IDs, and supporting facet IDs.
<code>qrels.parquet</code>	Query–chunk relevance labels projected from hidden memberships, including facet IDs, cluster roles, distractor types, and support type.
<code>embeddings*.npy</code> and <code>embeddings_metadata.json</code>	Chunk/query embedding arrays, IDs, model name, vector dimension, normalization flag, and device metadata.
<code>geometry_stats.parquet</code>	Per-query candidate-pool and geometry diagnostics, including <code>passes_filter</code> .
<code>evaluation_results.parquet</code>	Per-query, per-strategy, per- k , per- λ retrieval metrics.
<code>evaluation_stats.parquet</code>	Summary metrics grouped by strategy, k , and λ .
<code>embedding_geometry_points.parquet</code>	Per-query candidate coordinates, hidden labels, ranks, similarities, and strategy-selection flags for visualization.
<code>embedding_geometry_query_stats.parquet</code>	Per-query embedding-geometry diagnostics for plotted cases.

References

- [1] Zahra Abbasiantaeb, Simon Lupart, and Mohammad Aliannejadi. **Generating multi-aspect queries for conversational search**. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics*, pages 8201–8217, 2026. doi: 10.18653/v1/2026.eacl-long.383. URL <https://aclanthology.org/2026.eacl-long.383/>.
- [2] Cristian Cosentino, Annamaria Defilippo, Marco Dossena, Christopher Irwin, Sara Joubbi, and Pietro Liò. **Healthbranches: Synthesizing clinically-grounded question answering datasets via decision pathways**, 2025.
- [3] Ilias Driouich, Hongliu Cao, and Eoin Thomas. **Diverse and private synthetic datasets generation for RAG evaluation: A multi-agent framework**, 2025. URL <https://arxiv.org/abs/2508.18929>.
- [4] Albert Gong, Kamilė Stankevičiūtė, Chao Wan, Anmol Kabra, Raphael Thesmar, Johann Lee, Julius Klenke, Carla P. Gomes, and Kilian Q. Weinberger. **PhantomWiki: On-demand datasets for reasoning and retrieval evaluation**. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, 2025. URL <https://arxiv.org/abs/2502.20377>.
- [5] Yue Huang, Siyuan Wu, Chujie Gao, Dongping Chen, Qihui Zhang, Yao Wan, Tianyi Zhou, Chaowei Xiao, Jianfeng Gao, Lichao Sun, and Xiangliang Zhang. **DATAGEN: Unified synthetic dataset via large language models**. In *International Conference on Learning Representations*, 2025.
- [6] Priyanka Kargupta, Runchu Tian, and Jiawei Han. **Beyond true or false: Retrieval-augmented hierarchical analysis of nuanced claims**. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, pages 29664–29679, 2025. doi: 10.18653/v1/2025.acl-long.1434. URL <https://aclanthology.org/2025.acl-long.1434/>.
- [7] Tristan Kenneweg, Philip Kenneweg, and Barbara Hammer. **RAGVAL: Automatic dataset creation and evaluation for RAG systems**. In *2024 2nd International Conference on Foundation and Large Language Models*, pages 470–475, 2024. doi: 10.1109/FLLM63129.2024.10852482.
- [8] Seungone Kim, Juyoung Suk, Xiang Yue, Vijay Viswanathan, Seongyun Lee, Yizhong Wang, Kiril Gashteovski, Carolin Lawrence, Sean Welleck, and Graham Neubig. **Evaluating language models as synthetic data generators**. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, pages 6385–6403, 2025. URL <https://aclanthology.org/2025.acl-long.320/>.
- [9] Jeongsoo Lee, Daeyong Kwon, Kyohoon Jin, Junnyeong Jeong, Minwoo Sim, and Minwoo Kim. **MHTS: Multi-hop tree structure framework for generating difficulty-controllable QA datasets for RAG evaluation**, 2025. URL <https://arxiv.org/abs/2504.08756>.
- [10] Diego Pastrían, Nicolás Hidalgo, Víctor Reyes, and Erika Rosas. **Evolutionary multi-objective prompt learning for synthetic text data generation with black-box large language models**. *Applied Sciences*, 16(8):3623, 2026. doi: 10.3390/app16083623. URL <https://doi.org/10.3390/app16083623>.
- [11] Hossein A. Rahmani, Nick Craswell, Emine Yilmaz, Bhaskar Mitra, and Daniel Campos. **Synthetic test collections for retrieval evaluation**. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2647–2651, 2024. doi: 10.1145/3626772.3657942. URL <https://dl.acm.org/doi/10.1145/3626772.3657942>.
- [12] Haiyang Shen, Hang Yan, Zhongshi Xing, Mugeng Liu, Yue Li, Zhiyang Chen, Yuxiang Wang, Jiuzheng Wang, and Yun Ma. **DRAGON: Domain-specific robust automatic data generation for RAG optimization**. In *Findings of the Association for Computational Linguistics: EACL*, pages 1065–1078, 2026. doi: 10.18653/v1/2026.findings-eacl.56. URL <https://aclanthology.org/2026.findings-eacl.56/>.
- [13] Yuhong Sun, Joachim Rahmfeld, Chris Weaver, Weijia Chen, Roshan Desai, Wenxi Huang, and Mark H. Butler. **EnterpriseRAG-Bench: A RAG benchmark for company internal knowledge**, 2026. URL <https://arxiv.org/abs/2605.05253>.
- [14] Zafar et al. **KI-MAG: A knowledge-infused abstractive question answering system in medical domain**, 2024.
- [15] Angelo Ziletti and Leonardo D’Ambrosi. **Generating patient cohorts from electronic health records using two-step retrieval-augmented text-to-SQL generation**, 2025.